

ZaraiLink — Demo

Deep Dive

Working document for Salman Adnan

July 7, 2026

Honesty check

This document explains the **demo codebase only**: `zarailink-demo`, a scripted, fabricated-data product tour with zero live network calls. It does *not* document the private team project (ZaraiLink) whose screens it visually reproduces. That real project is a three-person final-year team build: a Django/DRF backend and React frontend with an NLU (Natural Language Understanding) pipeline, hybrid retrieval, and a learning-to-rank model over real Pakistani customs trade data. Salman Adnan led it (64 of 97 commits), but teammates Umar Kashif and Fahad Nadeem wrote real, substantial parts of it too — so publishing the full source is not a decision one team member can make alone without the others' consent. **This demo exists specifically so the real work can be shown to hiring panels without exposing the team's private source.** Every claim below is drawn directly from the demo project's own files (`README.md`, `index.html`, `app.js`, `demo-data.js`, `icons.js`, `styles.css`, `netlify.toml`, `LICENSE`) as they exist today. Nothing about the real system's internals, algorithms, or team members' individual contributions is asserted beyond what the demo's own README states.

Contents

1	What this project is, in one paragraph	3
1.1	Why a demo instead of the real thing	3
2	Beginner concepts used throughout this document	3
3	Directory layout	4
4	Architecture at a glance	5
5	The six views	5
6	File-by-file walkthrough	6
6.1	<code>index.html</code> — the page shell and every screen's markup	6
6.1.1	Dashboard view markup	7
6.1.2	Search Results (Data Dashboard) view markup	7
6.1.3	Trade Directory view markup	7
6.1.4	Watchlist view markup	7

6.1.5	Company Profile view markup	7
6.1.6	Subscription view markup	7
6.1.7	Modals	7
7	demo-data.js — the fabricated inputs	8
8	icons.js — a small hand-built icon library	8
9	app.js — routing, state, and every screen’s logic	9
9.1	Module-level state	9
9.2	View routing	9
9.3	Dashboard: live search-mode detection	9
9.4	Running a search	9
9.5	Trade-direction pills	10
9.6	Cards not yet unlocked: locking, not hiding	10
9.7	Trade Directory	10
9.8	Watchlist	11
9.9	Company Profile and the masking pattern	11
9.10	The unlock modal flow	11
9.11	Subscription	12
9.12	Global click handling and wiring	12
10	styles.css — visual design, read from the real app’s actual screens	13
11	netlify.toml, LICENSE, and fonts	13
12	Team-authorship honesty, stated explicitly	14
13	What was deliberately left out, and why	14
14	Development history worth noting	14
15	License, in plain terms	15
16	Summary table	15

1 What this project is, in one paragraph

`zarailink-demo` is a single static web page (no server, no build step, no framework) that behaves like a small multi-screen application: a Dashboard, a Search Results screen, a Trade Directory, a Watchlist, a Company Profile, and a Subscription page, all switched between with plain JavaScript inside one already-loaded HTML file. It reproduces the real ZaraiLink app's actual navigation and component structure — read directly from the real app's own React source files, named explicitly in the demo's own comments (e.g. `components/Dashboard/Dashboard.js`, `components/Search/DataDashboard.js`) — against four fabricated companies with fabricated contacts, products, and trade statistics. The centerpiece interaction is a token-gated “unlock” economy: revealing a company's contact details (phone, email, WhatsApp) costs one token per contact, mirrored faithfully from the real app's per-contact unlock design rather than a single all-or-nothing toggle.

1.1 Why a demo instead of the real thing

ZaraiLink is a real, three-person, final-year academic team project built around real Pakistani customs trade data, a Django/DRF (Django REST Framework, a Python web framework and its toolkit for building JSON APIs) backend, a React frontend, and machine-learning components (an NLU pipeline for parsing queries, hybrid retrieval, and a learning-to-rank model). Because two other people wrote real, substantial parts of the codebase, releasing the source publicly is a decision that needs every author's consent, not just the demo author's. Rather than wait indefinitely or publish without full consent, this demo reproduces the visible screens and interaction patterns — which are, after all, what any browser already renders the moment someone loads the real app — against invented data, so hiring panels have something concrete to click through today.

2 Beginner concepts used throughout this document

- **Static page:** a web page made only of HTML, CSS, and JavaScript files served as-is, with no server-side code generating content per request.
- **Single-page application (SPA) pattern, done without a framework:** normally, switching between app “screens” (like Dashboard vs Search Results) means loading a new HTML page from the server. An SPA instead keeps one HTML page loaded and swaps which section is visible using JavaScript. This demo does that manually: every screen already exists in `index.html` as a hidden `<section class="view">`, and a function called `switchView` shows one and hides the rest — the same visual effect a full SPA framework (like the real app's React) would produce, achieved here with plain DOM class toggling instead.
- **React / component:** a JavaScript framework (used by the real app, not this demo) for building UIs out of reusable, named pieces called components (e.g. `Dashboard.js`, `CompanyProfile.js`). This demo has no React and no build step; it reproduces what those components render by hand, in template-literal HTML strings.
- **DOM (Document Object Model):** the in-memory tree of elements a browser builds from an HTML page, which JavaScript reads and edits to change what's on screen.
- **Event delegation:** attaching one click listener to a large container (here, `document.body`) and inspecting which specific descendant element was actually clicked, rather than attaching a separate listener to every button individually.

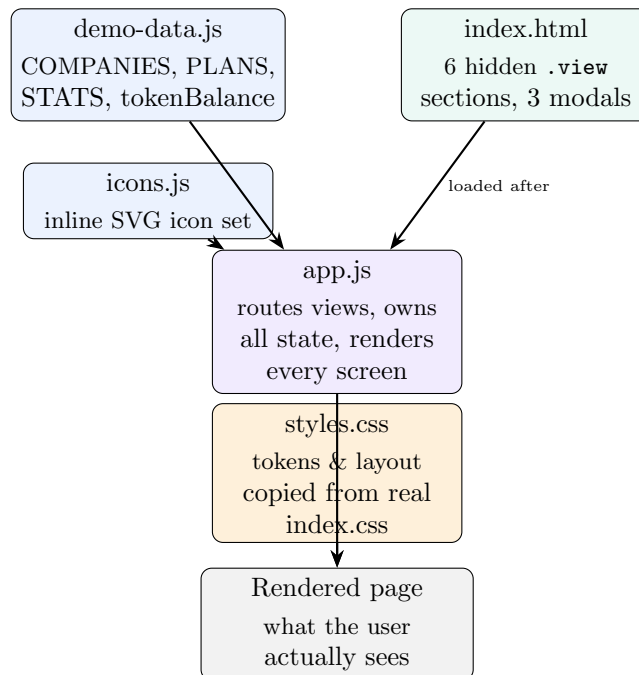
- **NLU (Natural Language Understanding)**: a category of techniques for extracting structured meaning from free-text queries (e.g. turning “buy urea from India” into a structured search). Used by the real app; not implemented in this demo, which only simulates recognizing three *modes* of query by simple pattern matching (see §9.3).
- **Hybrid retrieval / learning-to-rank**: real-app search techniques combining multiple signals (e.g. keyword matching and learned relevance models) to rank results. Not present in this demo; the demo’s “search” always returns the same four fabricated companies filtered by a fixed rule.
- **HS code**: a Harmonized System code, a standardized numeric classification used worldwide in customs and trade to identify product categories (e.g. 1702.3000 for a sugar/dextrose product). Real customs systems use these; the demo uses three fabricated examples for display only.
- **Token economy**: a common product pattern where users spend a limited, purchasable resource (“tokens”) to unlock premium actions. Here, unlocking one contact’s details costs 1 token; unlocking a whole result category costs a fixed amount taken from the fabricated balance.
- **Design tokens**: named, reusable CSS values (colors, radii, shadows) defined once and referenced throughout a stylesheet.
- **Variable font / woff2**: a single font file that can render a full range of weights (thin to black) instead of needing a separate file per weight; **woff2** is a compressed web font format. This demo self-hosts one (`assets/fonts/inter-variable.woff2`) rather than fetching it from Google Fonts, keeping the page fully offline.

3 Directory layout

```
zarailink-demo/
├── index.html    (all six views, hidden/shown by JS)
├── app.js        (view routing, all screen logic, unlock flow)
├── demo-data.js  (fabricated companies, plans, stats)
├── icons.js      (hand-drawn SVG icon set matching Lucide)
├── styles.css    (design tokens & per-screen layout, from real app's index.css)
├── netlify.toml  (hosting config)
├── README.md     (purpose, features, honesty notes)
├── LICENSE       (PolyForm Strict 1.0.0)
├── assets/
│   └── fonts/
│       └── inter-variable.woff2  (self-hosted Inter font)
```

Nine files, no dependency manager, no `package.json`, no build tool — everything the browser needs is these files as-is.

4 Architecture at a glance



As with the ReelFlow demo, this is a closed loop: every input is a JavaScript constant baked into `demo-data.js`, and every output is a DOM mutation inside the same tab. No arrow leaves the diagram.

5 The six views

`index.html` defines every screen as a `<section class="view" id="view-...">`. Only one carries the extra class `is-active` at a time; `styles.css` hides every `.view` by default (`display:none`) and shows only `.view.is-active`. Switching screens is therefore just moving that one class between sections — no page reload, no new HTML fetched.

View id	What it reproduces (per README, from the real component)
<code>view-dashboard</code>	Post-login search home: “4 Ways to Search” method card, live search-mode pill, Import/Export scope toggle, quick-suggestion pills, stats strip, features grid. Mirrors <code>components/Dashboard/Dashboard.js</code> .
<code>view-results</code>	Search results: sticky dark header, “Refine by Product” sidebar, four trade-direction pills, result cards, token-gated category unlock banner. Mirrors <code>components/Search/DataDashboard.js</code> .
<code>view-directory</code>	Trade Directory: searchable grid of company cards with country/sector/type badges and a watchlist star. Mirrors <code>components/TradeDirectory/FindSuppliers.js</code> / <code>FindBuyers.js</code> .
<code>view-watchlist</code>	Starred companies from the Trade Directory.
<code>view-profile</code>	Company Profile: stats grid, products, and the per-contact unlock UI (masked/blurred fields, seniority badge, confirm/success modal). Mirrors <code>components/TradeDirectory/CompanyProfile.js</code> .
<code>view-subscriptions</code>	Subscription plan cards with token allotments and feature lists. Mirrors <code>components/Subscriptions/Subscription.js</code> .

6 File-by-file walkthrough

6.1 `index.html` — the page shell and every screen’s markup

Structurally, top to bottom:

1. A `<meta name="robots" content="noindex">` tag (not meant to be search-indexed).
2. A yellow `.demo-banner`: *“Demo mode — fabricated example data. No live calls happen here; no real company or customs data is shown.”* — the first thing a visitor sees.
3. A dark, sticky `<nav class="zl-navbar">`: an emerald square logo mark with the letter Z, the “ZaraiLink” wordmark, nav links (Dashboard, an “Intelligence” dropdown containing Find Suppliers/Find Buyers, Subscription, Watchlist), a token-balance pill (`#token-balance`), a theme-toggle icon button (decorative only — no dark-mode logic is wired to it in `app.js`), and a gradient user avatar with a “Demo User” label.
4. A `<main>` containing all six `<section class="view">` blocks described above, each with its own internal structure (detailed per view below), plus three modal overlays (`#modal-confirm`, `#modal-success`, `#modal-insufficient`) sitting outside `<main>` so they can overlay any view.
5. A `<footer class="site-foot">` stating plainly: *“This is a companion demo for a private repository (team project, no current publishing consent from all authors). It contains no real company or customs data and makes no live calls; every result on this page is pre-scripted.”*
6. Three `<script>` tags at the end of `<body>`, in dependency order: `demo-data.js`, then `icons.js`, then `app.js`. Order matters for the same reason as in the ReelFlow demo — there is no module

system, so later scripts rely on earlier ones having already defined their global constants/functions.

6.1.1 Dashboard view markup

A hero section (`.dash-hero`) containing: an eyebrow label (`zarailink / demo`), the “4 Ways to Search ZaraiLink” method card with four colored tiles (HS Code / Category / Product Name / AI Query, each with a worked example like `1702.3000` or `buy urea from india`), a headline personalized with the fabricated user’s name, a search pill (icon + text input + Search button), an empty `#mode-pill-wrap` div that JavaScript fills in live as the user types, an Import/Export scope toggle, quick-suggestion pills, and an example query row. Below the hero: a dark stats strip (record/company/product counts) and a four-card features grid, then a bottom call-to-action banner linking to the Trade Directory.

6.1.2 Search Results (Data Dashboard) view markup

A sticky dark header showing the current query as a title, a shipment-count chip, and a “Dashboard” back-link chip. Below it, a two-column body: a left sidebar (`#dd-filters`, populated with one checkbox per unique product name found across all fabricated companies) and a main column holding the four trade-direction pills (`#dd-pills`) and a results container (`#dd-results-wrap`) that starts with a prompt to select a pill.

6.1.3 Trade Directory view markup

A header band with a title/lede that flips between “Find Suppliers” and “Find Buyers,” a search input, and a “Switch to Buyers/Suppliers” toggle button, followed by a company-card grid (`#directory-grid`).

6.1.4 Watchlist view markup

A header band and a company-card grid (`#watchlist-grid`), reusing the exact same card markup function as the Directory.

6.1.5 Company Profile view markup

A header with a back-link, a large avatar, the company name (with a “Verified” badge if applicable), legal name, and location, then a stats grid, a “Products Dealt In” pill row, and a “Key Contacts” section with a summary line (e.g. “1 of 2 contacts unlocked · 1 token each”) above the per-contact cards.

6.1.6 Subscription view markup

A header band and a three-card plan grid (`#plan-grid`).

6.1.7 Modals

Three fixed-position overlay `<div>`s, normally hidden (`display:none` via CSS until an `is-open` class is added): a confirm-unlock dialog, a success dialog showing the just-unlocked contact’s fabricated details, and an “insufficient tokens” dialog with a button that jumps to the Subscription view.

7 demo-data.js — the fabricated inputs

Five top-level pieces of data, none derived from any real customs record:

- **USER**: { `name`: "Demo User", `initial`: "D" }.
- **tokenBalance**: a mutable `let`, starting at 7 — this is the single running counter that every unlock action in the whole demo debits from.
- **HS_TREE**: three fabricated HS-code entries (Dextrose, Urea 46%, PVC Resin) with codes and categories — defined but, on inspection of `app.js`, not actually read anywhere at runtime; it exists as reference data matching the real app’s HS-code concept without being wired into a live filter in this demo.
- **COMPANIES**: an array of four fabricated companies (three in Brazil under a “Sweeteners” sector, one in Pakistan under “Fertilizers”), each with: `id`, `name`, `country`, `sector`, `type`, `legal name`, `district/province`, `founding year`, `employee-count band`, a fabricated `.example.com` website, verification flags, an array of **products** (`name`, `variety`, a “value added” flag), trade statistics (average price, volume, shipment count, average shipment size, last-active date), and an array of **contacts**. Each contact carries its own `id`, `name`, `designation`, an `is_public` flag, an `unlocked` flag (both start `false` except one deliberately pre-unlocked “public” contact), and fabricated `phone`, `email`, and `whatsapp` values.
- **SUBSCRIPTION_PLANS**: three plans (Starter, Growth “Popular”, Enterprise) each with an `id`, `name`, `token allotment`, `price`, `period`, and a feature map (some values `true/false`, some descriptive strings like “20 tokens included”).
- **STATS**: three display-only numbers for the Dashboard’s stats strip (“50,000+” records, “2,000+” companies, “500+” products) — round, clearly illustrative figures, not measurements of anything real.

The file’s own header comment states plainly: *“Fabricated demo data. No real companies, no real customs/trade data, no live calls. Structure and field names mirror the real app’s actual components ... so this demo represents the real product’s real screens.”* — i.e. the *shape* of the data (field names, nesting) is real; the *values* are invented.

Beginner note — why website domains end in `.example.com`: `example.com` (and `.example`, `.example.org`) are domain names permanently reserved by internet standards specifically for use in documentation and examples, guaranteed never to resolve to a real website. Using them here for fabricated company websites is a deliberate, standard way to signal “this is not a real address” without needing a disclaimer on every field.

8 icons.js — a small hand-built icon library

Defines one function, `icon(name, size)`, backed by a lookup table of SVG `<path>/<circle>` bodies for about 20 icon names (search, map-pin, lock, check-circle, star, phone, mail, and so on). Each is wrapped in an `<svg>` with a fixed 24x24 `viewBox`, 2px stroke width, and round line caps/joins — deliberately matching the visual signature of Lucide, the icon set the real app uses (per the file’s own comment). The file’s comment explains *why* this exists rather than falling back to emoji: a real screenshot of the app (`personal-website/assets/screenshots/zarailink.webp`,

referenced in the comment though not part of this project’s own files) showed that emoji “read as a completely different, lower-fidelity visual language” compared to the real app’s clean line icons, so hand-drawn SVGs matching the same style were used instead.

9 app.js — routing, state, and every screen’s logic

This is the largest and most complex file in the project. It is organized into clearly commented sections; each is covered in the order it appears.

9.1 Module-level state

Six variables declared at the top of the file hold *all* mutable state for the entire demo:

```
const watchedIds = new Set(); // company ids starred in the directory
let currentScope = "import"; // Dashboard's Import/Export toggle
let currentRole = "Supplier"; // Trade Directory: Supplier | Buyer
let currentProfileId = null; // which company profile is open
let ddUnlocked = false; // has the current result category been unlocked
let ddActivePill = null; // which trade-direction pill is selected
let pendingUnlock = null; // { companyId, contactId } awaiting modal confirm
```

Combined with each company object’s own `contacts[].unlocked` flags and the global `tokenBalance` from `demo-data.js`, this is the entire state model of the app — again, all in browser memory, all lost on refresh.

9.2 View routing

`switchView(viewId)` toggles the `is-active` class on the matching `.view` section and the matching nav link, then smooth- scrolls to the top of the page. `renderTokenBalance()` writes the current `tokenBalance` into every element with id `token-balance` (there is only one, in the navbar, but the code is written generically with `querySelectorAll`).

9.3 Dashboard: live search-mode detection

`detectMode(query, scopeSelected)` implements a simple, three-way rule, matching the real Dashboard’s detection logic per the README:

1. If the trimmed query starts with a digit, mode is `"hscode"`.
2. Otherwise, if a scope (Import/Export) has been selected and the query is non-empty, mode is `"ai"`.
3. Otherwise, mode is `"product"`.

`renderModePill(query)` calls this on every keystroke (wired to the search input’s `input` event) and renders a colored pill labeled “HS Code Mode,” “AI Query Mode,” or “Product Search” beneath the search bar — purely a live label; it does not change what `runSearch` actually returns.

9.4 Running a search

`runSearch(query)` always operates on the same fixed `COMPANIES` array. It resets the two results-view flags (`ddUnlocked = false`, `ddActivePill = null`), sets the results header title (a fixed

HS-code label if in HS-code mode, otherwise the raw query text) and subtitle, computes a shipment-count chip as `COMPANIES.length * 14` (a display-only fabricated multiplier, not a count of anything real), rebuilds the sidebar filter checkboxes, rebuilds the four trade-direction pills, and switches to the Results view with a placeholder “select a trade perspective” message until the user picks a pill.

9.5 Trade-direction pills

Four fixed pill definitions:

```
{ id: "foreign-suppliers", label: "Foreign Suppliers", filter: c => c.country !== "Pakistan" }
{ id: "foreign-buyers", label: "Foreign Buyers", filter: c => c.country !== "Pakistan" }
{ id: "pk-buyers", label: "Pakistani Buyers", filter: c => c.country === "Pakistan" }
{ id: "pk-suppliers", label: "Pakistani Suppliers", filter: c => c.country === "Pakistan" }
```

Note that `foreign-suppliers` and `foreign-buyers` use the *identical* filter predicate (as do the two Pakistani pills), so in this demo’s fabricated dataset, clicking either of a same-country pair shows the same three or one company; the four labels exist to mirror the real app’s four-way Import/Export × Foreign/Pakistani framing even though, with only four fabricated companies, the demo cannot meaningfully distinguish buyer role from supplier role within a country. This is a faithful reproduction of the real UI’s structure, not a claim that the demo implements the real role-distinguishing logic.

`selectPill(pillId)` filters `COMPANIES` by the chosen pill’s predicate, shows a result count, renders one `.dd-card` per matching company (via `renderDdCards`), and — if the category has not yet been unlocked — appends an “Unlock Category Intelligence” banner. That banner’s button spends 5 tokens (despite the button label reading “Unlock for 500 Tokens” — a mismatch between the displayed copy and the actual deduction in the code, worth flagging explicitly since it is a real discrepancy in the demo as written) if `tokenBalance >= 5`, otherwise opens the insufficient-tokens modal.

Flagged discrepancy: the unlock-category button’s visible text says “*Unlock for 500 Tokens*” but the click handler’s guard and deduction both use 5, not 500. This is a copy/logic mismatch in the demo’s own source as it exists today, not a claim about the real system; it is called out here rather than silently smoothed over, per this document’s no-fabrication rule.

9.6 Cards not yet unlocked: locking, not hiding

`renderDdCards(results)` renders each result as a `.dd-card`; while the category is locked (`ddUnlocked === false`), the card gets an `is-locked` class, which `styles.css` turns into a blurred, non-clickable action area (`filter: blur(2px); pointer-events: none;`) — the company’s name and stats stay visible, but the “View Deal” action is blurred and inert until the category is unlocked.

9.7 Trade Directory

`renderDirectory()` sets the title/lede/switch-button text based on `currentRole` (“Supplier” or “Buyer”), filters `COMPANIES` by a case-insensitive substring match against the search box, and renders one `companyCardHtml(c)` per match. Note this directory view does not actually filter by

supplier-vs-buyer role in the underlying data (the fabricated companies carry no such role field) — the `currentRole` toggle changes labels and the “switch” button text, consistent with the demo’s own stated scope of reproducing screen structure and interaction, not a fully role-aware dataset.

`companyCardHtml(c)` renders an avatar (the company name’s first letter), a star/watch toggle button whose filled/outline state depends on `watchedIds`, badges for country/sector/type, a “Verified Contacts” badge shown only when both `has_key_contacts` and `verified` are true, and a “View Profile & Contacts” button.

9.8 Watchlist

`renderWatchlist()` filters `COMPANIES` to those whose id is in `watchedIds` and reuses the exact same `companyCardHtml` function as the Directory — one card renderer, two different filters feeding it.

9.9 Company Profile and the masking pattern

`getSeniorityBadge(designation)` classifies a contact’s free-text job title into one of five badge categories by case-insensitive regular-expression matching, in this priority order: `ceo|chief|director|owner|partner|president|founder` → “Decision Maker”; `procurement` → “Procurement”; `sales` → “Sales”; `manag|head` → “Management”; anything else → “Team Member.”

`maskEmail(email)` keeps the first character of the local part and the whole domain, replacing the rest with five bullet characters (e.g. `e.....@example.com`). `maskPhone(phone)` keeps the first three characters (typically a country-code prefix) and replaces the rest with a fixed “`.....`” pattern. Locked contact rows additionally get a CSS `blur(3px)` filter on top of this text masking, so a locked field is both textually obscured *and* visually blurred — two independent layers of “you can’t read this yet.”

`openProfile(id)` looks up the company, fills in the header (avatar, name with an optional “Verified” badge, legal name if different from the display name, location), computes an unlocked-contact count, fills the stats grid, renders product pills, calls `renderContacts(c)`, and switches to the Profile view.

`renderContacts(c)` writes a summary line (“No contacts listed,” “All N contacts unlocked,” or “M of N contacts unlocked · 1 token each”), then renders one `.contact-card` per contact. A contact counts as unlocked if either its `unlocked` flag is true *or* its `is_public` flag is true — the fabricated dataset includes exactly one contact (Cascade Commodities Group’s Priya Nair) marked `is_public: true`, so she always displays real, unmasked details without ever costing a token, distinguishing “already public” from “unlocked by spending a token.” Unlocked cards get a green avatar and (if not simply public) an “Unlocked” badge; locked cards get a grey avatar and a “Unlock Contact Details · 1 token” button. If any contact on the page is still locked and the user’s token balance is below 1, an extra note box appears linking to the Subscription view.

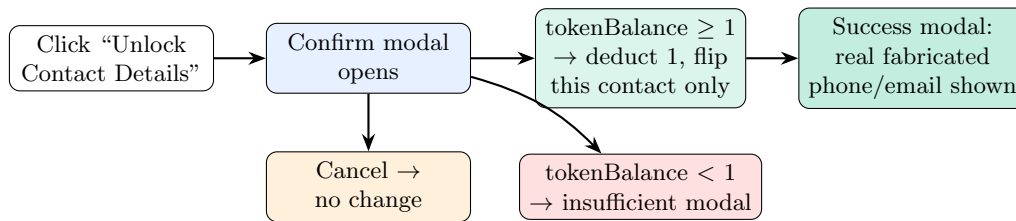
9.10 The unlock modal flow

Three tiny helpers, `showModal(id)/hideModal(id)`, add or remove the `is-open` class that `styles.css` uses to show a modal overlay. The actual flow:

1. `requestUnlock(contactId)`: looks up the current profile’s matching contact, stores { `companyId`,

`contactId` } in the module- level `pendingUnlock`, fills the confirm dialog’s text with the contact’s name and the current token balance, and opens `#modal-confirm`.

2. User clicks Cancel → `pendingUnlock` is cleared and the modal closes with no state change. User clicks “Unlock (1 token)” → `confirmUnlock()` runs.
3. `confirmUnlock()` hides the confirm modal; if `pendingUnlock` is unset, it returns immediately (defensive guard). If `tokenBalance < 1`, it opens the insufficient-tokens modal instead and clears `pendingUnlock` without charging anything. Otherwise it decrements `tokenBalance` by 1, sets that specific contact’s `unlocked` flag to `true`, re-renders the token pill and the contact list (so only that one contact flips to unmasked — every other locked contact on the same page stays locked), fills the success dialog with the now-real name/phone/email, and opens it.



9.11 Subscription

`renderPlans()` renders the three `SUBSCRIPTION_PLANS` as cards, marking the “popular” one with a badge, listing each plan’s feature map (boolean features show just the label; string-valued features show `label: value`), and a “Get Plan” button.

9.12 Global click handling and wiring

A single `document.body` click listener (event delegation, as in the ReelFlow demo) inspects the clicked element for a series of data attributes in priority order and dispatches accordingly:

Attribute matched	Action
<code>[data-view]</code>	Switch view (and set <code>currentRole</code> if the link carries <code>data-role</code>); re-render Directory.
<code>[data-open-profile]</code>	<code>openProfile(id)</code> .
<code>[data-watch]</code>	Toggle that company’s id in <code>watchedIds</code> ; re-render the Directory.
<code>[data-unlock-contact]</code>	<code>requestUnlock(id)</code> .
<code>[data-pill]</code>	<code>selectPill(id)</code> .
<code>[data-buy-plan]</code>	Add that plan’s token allotment to <code>tokenBalance</code> , flash the button to “Token
<code>#intel-toggle</code>	Toggle the Intelligence nav dropdown open/closed.
click outside <code>#intel-dropdown</code>	Close the dropdown.

Beyond the single delegated listener, a handful of direct listeners are attached once at `DOMContentLoaded`: the query input’s live mode-pill update and Enter-to-search, the Import/Export scope buttons, the example- query buttons (which fill the search box and update the mode pill without actually running a search), the Directory’s role-switch button and live search-as-you-type, and the three modal buttons (cancel/confirm/close). The initializer also seeds the Dashboard’s username/avatar/stats text, renders the initial token balance and plan cards, and calls `renderDirectory()` once so the Directory has content the first time it’s visited.

10 styles.css — visual design, read from the real app’s actual screens

The stylesheet’s opening comment is explicit about method: the structure and tokens “*reproduce the real app’s actual screens and components as closely as a static demo can, read directly from its source*” — and it names the specific real files each part of the CSS is reproducing (the navbar from `Navbar.js`, the Dashboard hero from `Dashboard.js`, the Data Dashboard from `DataDashboard.js`, directory cards from `FindSuppliers.js`, and the profile/unlock UI from `CompanyProfile.js`). Colors are stated to be copied 1:1 from the real app’s own `index.css` `:root` block. The same distinction drawn in the ReelFlow demo is drawn here too: this is the visual design language any browser rendering the real app already shows, not its private backend logic — the NLU pipeline, ranking model, real customs data, or actual API routes, none of which appear anywhere in this repo.

Key token groups:

Token	Role
--primary (#10B981), --primary-hover, --primary-light	emerald brand green, three shades
--blue, --violet, --amber	accent hues (used for method-card tiles, badges)
--slate-900/800/700/400	dark navbar and dark-band backgrounds/text
--bg, --bg-2, --bg-3	light page background layers (this app is light-themed, unlike ReelFlow’s dark theme)
--text, --text-2, --text-3	three text-contrast levels
--border, --border-2	hairline border colors
--shadow-sm/md/lg	three shared drop-shadow sizes
--r-card, --r-btn, --r-sm	three corner-radius sizes
--wrap	max content width (1200px, wider than ReelFlow’s 880px, for multi-column dashboard layouts)

Unlike `reelflow-studio-demo`’s single dark theme, this stylesheet is predominantly light (white/near-white surfaces) with a dark slate navbar and two dark full-width bands (the Dashboard’s stats strip and bottom CTA, and the Search Results sticky header) — matching a light-mode SaaS dashboard aesthetic rather than an all-dark console. The file is organized by screen, in page order: navbar, page-level scaffolding, Dashboard/search home, Data Dashboard, Trade Directory, Company Profile, modal, Subscription, footer — each section headed by a comment naming which real screen it reproduces.

11 netlify.toml, LICENSE, and fonts

`netlify.toml` is identical in shape to the ReelFlow demo’s: one `[build]` block publishing the repository root as-is. `LICENSE` is the same PolyForm Strict License 1.0.0 (see §15 for a plain-language summary). `assets/fonts/inter-variable.woff2` is a self-hosted copy of the Inter variable font, loaded via an `@font-face` rule with `font-weight: 100 900` (declaring it can render the entire

weight range from a single file) and `font-display: swap` (show a fallback system font immediately, swap to Inter once it finishes loading, rather than leaving text invisible while waiting).

12 Team-authorship honesty, stated explicitly

Because this is the more sensitive of the two projects being documented here, it is worth restating plainly, in this document’s own words, exactly what the demo’s README already discloses: ZaraiLink was a three-person team’s final-year project. Salman Adnan led it and wrote the majority of the commits (64 of 97), but the remaining commits represent real, substantial work by two named teammates, Umar Kashif and Fahad Nadeem. None of their code, their names as contributors to specific features, or any real project artifact appears in this demo repository. What is shown here is Salman Adnan’s own reconstruction of the *visible screens* — markup, CSS, and scripted interaction logic — built from scratch in this separate, public demo repository, not copied from or built on top of the private team repository’s actual source files.

13 What was deliberately left out, and why

- **No real NLU, retrieval, or ranking.** Search always returns the same four fabricated companies, filtered by simple, hard-coded rules (country equality for the trade-direction pills; substring match for the Directory search) — not the real app’s NLU pipeline, hybrid retrieval, or learning-to-rank model.
- **The Trade Intelligence cluster is absent.** The README’s own “What I’d do differently” section names this directly: the real app’s link-prediction, node2vec similar-company matching, and GNN (Graph Neural Network — a machine-learning model architecture designed to learn from graph-structured data, such as a network of companies and their trade relationships) graph work are called the product’s most technically distinctive piece, and are not represented in this demo at all; only the search, directory, profile, and subscription screens are.
- **Buyer/supplier role is UI-only in the Directory.** The role toggle changes labels and copy but the fabricated dataset carries no actual buyer-vs-supplier field to filter on.
- **No real trade data, company names, or customs records.** All four companies, their statistics, and their contacts are invented, using `.example.com` domains and fabricated Brazilian/Pakistani addresses.

14 Development history worth noting

The README’s “Challenges” section records that an earlier version of this demo matched only the real app’s *color palette* but invented its own, simpler single-flow layout, and that this was recognized as not reading like the real product; the current version was rebuilt screen-by-screen from the actual component source instead. The lesson recorded in the README’s own words: *“Matching a product’s color palette is a much lower bar than matching its actual information architecture.”*

15 License, in plain terms

PolyForm Strict License 1.0.0: you may run and study the software, and noncommercial, personal, and educational use is broadly permitted, but the license does not grant rights to redistribute the software or to create and distribute modified/derivative works from it, and it grants no right to sublicense the code to others.

16 Summary table

Aspect	This demo
Backend	None. Everything runs client-side in the browser.
Data source	<code>demo-data.js</code> : 4 fabricated companies, 3 subscription plans, fixed stats.
State storage	Module-level JS variables + per-contact <code>unlocked</code> flags, all lost on refresh.
“Search”	Fixed dataset filtered by simple, hard-coded rules; no NLU, retrieval, or ranking.
Unlock economy	Per-contact (1 token) and per-category (5 tokens, mislabeled “500” in the UI copy) deductions from a starting balance of 7.
Views	6 (Dashboard, Search Results, Trade Directory, Watchlist, Company Profile, Subscription) toggled by CSS class, no page reloads.
Styling	Light SaaS-dashboard theme, tokens copied verbatim from the real app’s <code>index.css</code> .
Fonts/icons	Self-hosted Inter variable font; hand-drawn SVG icons matching Lucide’s style.
Hosting	Static file publish via Netlify.
License	PolyForm Strict 1.0.0.
Why it exists	Team project; full-source publication needs all three authors’ consent, not yet given.